

HW9

Sri Srinivasan

January 2026

Problem 1

(a)

(i)

The softmax similarity can be rewritten using a Gaussian kernel, so

$$\exp\left(\frac{q^\top k}{\sqrt{D}}\right) = \exp\left(\frac{\|q\|^2}{2\sigma^2}\right) \exp\left(-\frac{\|q - k\|^2}{2\sigma^2}\right) \exp\left(\frac{\|k\|^2}{2\sigma^2}\right),$$

with $\sigma^2 = \sqrt{D}$. The middle factor corresponds to the Gaussian kernel

$$K_{\text{Gauss}}(q, k) = \exp\left(-\frac{\|q - k\|^2}{2\sigma^2}\right).$$

(ii)

Using random Fourier features $\phi_{\text{rand}}(\cdot)$ to approximate the Gaussian kernel,

$$K_{\text{Gauss}}(q, k) \approx \phi_{\text{rand}}(q)^\top \phi_{\text{rand}}(k),$$

the softmax similarity gives the approximation

$$\exp\left(\frac{q^\top k}{\sqrt{D}}\right) \approx \exp\left(\frac{\|q\|^2}{2\sigma^2}\right) \phi_{\text{rand}}(q)^\top \phi_{\text{rand}}(k) \exp\left(\frac{\|k\|^2}{2\sigma^2}\right).$$

Define

$$A = \exp\left(\frac{\|Q\|^2}{2\sigma^2}\right) \phi_{\text{rand}}(Q), \quad B = \phi_{\text{rand}}(K) \exp\left(\frac{\|K\|^2}{2\sigma^2}\right).$$

Then the attention output can be computed as

$$V' = \frac{A(B^\top V)}{A(B^\top \mathbf{1})}.$$

Computing random features is $O(ND_{\text{rand}}D)$, while the matrix multiplications need $O(ND_{\text{rand}}M)$, and the total complexity is

$$O(N(D + M)D_{\text{rand}}).$$

(b)

With causal masking, attention is restricted to indices $j \leq i$, and

$$V'_i = \frac{\sum_{j=1}^i \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^i \text{sim}(Q_i, K_j)}.$$

Applying kernel linearization gives

$$V'_i = \frac{\phi(Q_i)^\top \sum_{j=1}^i \phi(K_j) V_j^\top}{\phi(Q_i)^\top \sum_{j=1}^i \phi(K_j)}.$$

With the quantities

$$S_i = \sum_{j=1}^i \phi(K_j) V_j^\top, \quad Z_i = \sum_{j=1}^i \phi(K_j).$$

Then,

$$V'_i = \frac{\phi(Q_i)^\top S_i}{\phi(Q_i)^\top Z_i}.$$

Thus, the overall complexity is linear in the sequence length.

Problem 2

(a)

Passing the encoder hidden state directly into each decoder timestep removes the decoder's ability to condition on the full source sequence and restricts access to later encoder information, which is problematic when source and target word orders differ.

(b)

During training with teacher forcing, the decoder inputs are

$$\langle \text{SOS} \rangle, I, \text{ see}, a, \text{ dog}.$$

(c)

At evaluation time, the decoder uses its own previous predictions, so the inputs are

$$\langle \text{SOS} \rangle, I, \text{ saw}, a, \text{ dog}.$$

Problem 7

(a)

We are given one training point (x, y) , a true feature $\phi_1(x) = \phi(x)$, and an alias feature $\phi_2(x) = \phi(x)$. The learned function is

$$\hat{f}_{\beta, c}(x) = \beta_0 c_0 \phi(x) + \beta_1 c_1 \phi(x).$$

The minimal-norm regression problem is

$$\min_{\beta} \|\beta\|_2^2 \quad \text{s.t.} \quad y = (\beta_0 c_0 + \beta_1 c_1) \phi(x).$$

Among all solutions satisfying the constraint, the minimum-norm solution aligns β with (c_0, c_1) , giving

$$\hat{\beta} = \frac{y}{\phi(x)} \frac{1}{c_0^2 + c_1^2} c_0 c_1.$$

Thus,

$$\hat{\beta}_0 = \frac{c_0}{c_0^2 + c_1^2}, \quad \hat{\beta}_1 = \frac{c_1}{c_0^2 + c_1^2}.$$

(b)

Using the coefficients from part (a), the learned predictor is

$$\hat{f}(x) = \frac{c_0^2}{c_0^2 + c_1^2} \phi_1(x) + \frac{c_1^2}{c_0^2 + c_1^2} \phi_2(x).$$

The expected test error is

$$\mathcal{E} = E_{x_{test}} \left[\frac{1}{2} (\phi_1(x) - \hat{f}(x))^2 \right].$$

Since ϕ_1, ϕ_2 are orthonormal under the test distribution,

$$\mathcal{E} = \frac{1}{2} \left(1 - \frac{c_0^2}{c_0^2 + c_1^2} \right)^2 + \frac{1}{2} \left(\frac{c_1^2}{c_0^2 + c_1^2} \right)^2 = \frac{c_1^4}{(c_0^2 + c_1^2)^2}.$$

Taking gradients,

$$\frac{\partial \mathcal{E}}{\partial c_0} = -\frac{4c_0c_1^4}{(c_0^2 + c_1^2)^3}, \quad \frac{\partial \mathcal{E}}{\partial c_1} = \frac{4c_1^3c_0^2}{(c_0^2 + c_1^2)^3}.$$

This induces a rotational flow that drives $c_1 \rightarrow 0$ while preserving scale.

(c)

Running gradient descent on (c_0, c_1) with a sufficiently small step size shows that trajectories converge to

$$c_0 \rightarrow \|c^{(0)}\|_2, \quad c_1 \rightarrow 0.$$

As a result,

$$\hat{\beta}_0 \rightarrow \frac{1}{\|c^{(0)}\|_2}, \quad \hat{\beta}_1 \rightarrow 0,$$

meaning the learner assigns all weight to the true feature and suppresses the alias.

Problem 8

(a)

Human reaction time to form a coherent judgment is on the order of 10^{-2} seconds. This corresponds to neural processing across roughly 10^2 sequential stages. Modern large language models operate with a comparable depth, so assume depth is not the limiting factor. As given, human brain has approximately 10^{15} synapses. If each synapse stores on the order of 5 bits of information, this corresponds to

$$5 \times 10^{15} \text{ bits} \approx 6 \times 10^{14} \text{ bytes}.$$

A 16-bit float parameter stores 2 bytes, so matching this capacity requires on the order of

$$\frac{6 \times 10^{14}}{2} \approx 3 \times 10^{14}$$

parameters. Rounding, we see that human-level performance would plausibly require between 10^{14} and 10^{15} parameters.

(b)

The loss is modeled as

$$L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_0,$$

subject to the compute constraint

$$C = C_0 N D.$$

Solving for $N = \frac{C}{C_0 D}$ and substituting,

$$L(D) = A \left(\frac{C_0 D}{C} \right)^\alpha + \frac{B}{D^\beta} + L_0.$$

Taking the derivative with respect to D and setting it to zero yields

$$\alpha A \left(\frac{C_0}{C} \right)^\alpha D^{\alpha-1} = \beta B D^{-\beta-1}.$$

Solving for D gives

$$D_{opt}(C) = \left(\frac{\beta B}{\alpha A} \right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{C_0} \right)^{\frac{\alpha}{\alpha+\beta}},$$

and therefore

$$N_{opt}(C) = \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{C_0} \right)^{\frac{\beta}{\alpha+\beta}}.$$

Substituting,

$$\alpha = 0.34, \quad \beta = 0.28, \quad A = 406.4, \quad B = 410.7,$$

we have the approximations

$$N_{opt}(C) \approx 0.6 C^{0.45}, \quad D_{opt}(C) \approx 0.3 C^{0.55},$$

and

$$L_{opt}(C) \approx 1070 C^{-0.154} + 1.7.$$

(c)

A direct statistical fit reported in the same work yields a simplified optimal scaling,

$$N_{opt}(C) = 0.1 C^{0.5}, \quad D_{opt}(C) = 1.7 C^{0.5}.$$

(d)

Assume 1.4 tokens per English word. Then

$$10^{13} \text{ tokens} \Rightarrow \frac{10^{13}}{1.4} \approx 7.1 \times 10^{12} \text{ words}.$$

Each book contains $400 \times 300 = 1.2 \times 10^5$ words, so

$$\frac{7.1 \times 10^{12}}{1.2 \times 10^5} \approx 6 \times 10^7$$

books, so roughly sixty million books.

(e)

Each parameter uses 16 bits = 2 bytes.

$$10^9 \text{ params} \Rightarrow 2 \text{ GB}, \quad 10^{12} \text{ params} \Rightarrow 2 \text{ TB}.$$

An A100 has 40 GB of VRAM, so storing 2 TB requires

$$\frac{2000}{40} = 50$$

GPUs. With Adam-style optimizers requiring three additional parameter-sized buffers, the requirement increases by a factor of 4, to about 200 GPUs.

(f)

Using 2018 prices, SSD storage costs \$0.00015/MB and DRAM costs \$0.0059/MB.

$$2 \text{ TB} = 2 \times 10^6 \text{ MB}.$$

Thus,

$$SSDcost \approx 300 \text{ USD/year}, \quad DRAMcost \approx 12,000 \text{ USD/year}.$$

(g)

The model size is 2 TB and A100 bandwidth is 1.6 TB/s, so the minimum forward-pass latency is

$$\frac{2}{1.6} \approx 1.25 \text{ s}.$$

Autoregressive decoding yields one token per pass, or about

$$\frac{60}{1.25} \approx 46$$

tokens per minute. GPT-3 with 175B parameters is $\frac{1000}{175} \approx 5.7$ times faster, or about 260 tokens per minute.

(h)

One petaFLOP-day equals

$$10^{15} \text{ FLOP/s} \times 86400 \text{ s} = 8.64 \times 10^{19} \text{ FLOP}.$$

An A100 sustains 0.3 petaFLOP/s, so one petaFLOP-day equals 80 A100-hours. At \$2/hour, this costs \$160.

GPT-3 required 3640 petaFLOP-days, giving

$$3640 \times 160 \approx 6 \times 10^6 \text{ USD}.$$

The hypothetical GPT-5 requires 10^{26} FLOPs, which is

$$\frac{10^{26}}{8.64 \times 10^{19}} \approx 1.2 \times 10^6$$

petaFLOP-days, costing roughly \$160 million at full utilization. Accounting for realistic efficiency and restarts pushes the total to order \$1 billion.

With 10,000 A100 GPUs, the wall-clock training time is

$$\frac{8.3 \times 10^7 \text{ A100-hours}}{10^4} \approx 1 \text{ year}.$$

(j)

Inference uses 2 FLOPs per parameter per token. For GPT-3.5,

$$2 \times 1.75 \times 10^{11} \times 10^6 \approx 4 \times 10^{17} \text{ FLOP}.$$

Since one A100-hour provides 8.64×10^{19} FLOP, this costs about

$$\frac{4 \times 10^{17}}{8.64 \times 10^{19}} \approx 0.01 \text{ USD}.$$

At \$2 per million tokens, recouping \$10 million requires

$$\frac{10^7}{2} \times 10^6 = 5 \times 10^{12} \text{ tokens}.$$

This corresponds to $\frac{5 \times 10^{12}}{1400} \approx 4 \times 10^9$ essays of length 1000 words.

(k)

The Landauer limit gives $E_{bit} = k_B T \ln 2 \approx 3 \times 10^{-21}$ J. One FLOP erases 32 bits, so

$$E_{FLOP} \approx 10^{-19} \text{ J}.$$

At 300 TFLOP/s, the minimal power is

$$3 \times 10^{14} \times 10^{-19} = 3 \times 10^{-5} \text{ W},$$

far below the actual 300 W consumption.

(l)

GPT-3 emits 552 tCO₂. At \$112 per ton, this costs

$$552 \times 112 \approx 6.2 \times 10^4 \text{ USD},$$

which is about 1% of the training compute cost. A quarter-pound burger emits about 5.6 kg CO₂, so

$$\frac{5.52 \times 10^5}{5.6} \approx 9 \times 10^4$$

burgers produce comparable emissions.

Problem 9

(a)

The authors consider two neural network architectures. The first is a large deep convolutional network composed of several convolutional and pooling layers followed by fully connected layers. The second architecture is based on the Inception model introduced by Szegedy et al., which uses parallel convolutions of different sizes to improve computational efficiency and representation quality.

(b)

The triplet loss operates on triplets of images consisting of an anchor, a positive example of the same identity, and a negative example of a different identity. The objective enforces that the anchor is closer to the positive than to the negative by at least a fixed margin. Unlike the typical cross-entropy, triplet loss does not directly predict class labels and instead learns a similarity-preserving embedding space.

(c)

Generating all possible triplets is infeasible as the number of triplets grows cubically with the dataset size, and most triplets are uninformative. The authors address this by using online triplet mining within each mini-batch, choosing only informative triplets that violate or nearly violate the margin constraint.

(d)

The FaceNet models use approximately 140M parameters for the larger network (NN1) and 7.5M parameters for the Inception-based network (NN2), with roughly 1.6B FLOPs. In comparison, ResNet-50 has about 26M parameters and requires approximately 3.8B FLOPs, making it more expensive despite having fewer parameters.

(e)

Semi-hard negatives are negative examples that are farther from the anchor than the positive example but still fall within the margin. These are informative enough to allow for learning while avoiding the instability that can occur when using the hardest negatives.

(f)

Harmonic embeddings are those whose distances remain meaningful and comparable across different datasets or training conditions. This allows embeddings learned on separate datasets to be used together for face comparison and recognition.

(g)

Performance improves slightly when increasing the embedding dimensionality from 64 to 128, but degrades when increased further. The authors suggest that larger embedding spaces may require more training to converge and that the observed differences may be statistically insignificant.

(h)

Validation performance improves with increasing amounts of training data, though the gains diminish at larger scales. A tenfold increase from 2.6M to 26M images yields a significant improvement, while an additional tenfold increase results in only marginal gains, indicating performance saturation.

(i)

A notable emergent property of the learned embeddings is that faces naturally cluster by identity without explicit clustering supervision, with distances in the embedding space reflecting meaningful visual similarity.

(j)

The most interesting aspect of the paper is the use of online triplet mining with semi-hard negatives. This approach effectively balances training difficulty and stability, enabling scalable training on massive datasets and influencing later work in metric learning beyond face recognition.